

MICRO ENTREPRENEUR LEDGER APP

A Mobile-First Digital Bookkeeping Solution for Small Vendors

PHASE - 1 REPORT

SUBMITTED BY

MANOJ KUMAR S 230701178

MOHAMED IKRAM KHD 230701188

KAMALESH L 230701136

MITHUN AR 230701186

*in partial fulfilment for the award of the degree
of*

**BACHELOR OF ENGINEERING IN
COMPUTER SCIENCE AND ENGINEERING**



**RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI ANNA
UNIVERSITY, CHENNAI 600 025**

October 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled "**MICRO ENTREPRENEUR LEDGER APP: A MOBILE-FIRST DIGITAL BOOKKEEPING SOLUTION FOR SMALL VENDORS**" is the bonafide work of **MANOJ KUMAR S (230701178)**, **MOHAMED IKRAM KHD (230701188)**, **KAMALESH L (230701136)** and **MITHUN AR (230701186)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE

Dr. J. Manoranjini

Professor & Head
Department of Computer Science and
Engineering,
Rajalakshmi Engineering
College, Thandalam, Chennai - 602105.

SIGNATURE

Ms. S. Sathiyavathi

Professor
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College,
Thandalam, Chennai - 602105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Micro and small businesses such as kirana stores, street-vending units, tea shops, local retailers, and home-based enterprises frequently maintain daily sales, expenses, customer dues, and supplier payments in paper notebooks. Although familiar, paper ledgers are difficult to search, easy to lose or damage, and depend heavily on manual calculation. The **Micro Entrepreneur Ledger App** addresses this problem with a simple digital bookkeeping platform aligned with United Nations Sustainable Development Goal 8, particularly the objective of supporting entrepreneurship and the growth of micro and small enterprises through better access to financial tools.

The current Phase-1 prototype is implemented as a full-stack web application using **Next.js 14** and **React 18**, with Tailwind CSS for the interface, Next.js Server Actions and Route Handlers for application logic, and **Prisma ORM over SQLite** for structured ledger persistence. The system records customers and suppliers, credit and payment transactions, sales, purchases, and operating expenses; continuously derives balances and summaries; and presents daily and monthly views through dashboards and Recharts visualizations. Authentication is handled through NextAuth.js with bcryptjs-based password hashing.

To reduce typing effort for non-technical users, the prototype also incorporates the browser Web Speech API with a custom heuristic natural-language parser for English, Tamil-oriented phrases, and Tanglish-style input. A spoken command is converted into structured fields such as transaction type, party, amount, and note, then shown for confirmation before storage. Invoice support can generate printable bills and UPI payment QR codes through standard UPI URI formatting, while report features support CSV export, HTML5 printing, and PDF-oriented output.

The project emphasizes understandable financial records rather than complex accounting terminology. Core design choices therefore include minimal data entry, prominent actions, traceable transaction history, clear debit/credit meaning, search and filtering, and visible due amounts. The prototype demonstrates how a lightweight digital ledger can combine bookkeeping, assistive voice entry, payment support, and financial summaries in one interface. Cloud synchronization, deeper offline-first packaging, GST-compliant invoicing, and predictive analytics remain appropriate extensions for later phases after the core ledger workflow is validated with representative micro-entrepreneur users.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E., F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. MANORANJINI**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our Project Guide, **Ms. S. SATHIYAVATHI, Professor**, Department of Computer Science and Engineering, for her valuable guidance throughout the survey, design, and implementation phases of this project.

We also extend our sincere thanks to all the faculty members of the Department of Computer Science and Engineering, Rajalakshmi Engineering College, for their support and encouragement. Finally, we express our heartfelt gratitude to our parents, friends, and well wishers for their constant motivation and support throughout the successful completion of this project.

TABLE OF CONTENTS

Section No.	Title	Page No.
CHAPTER 1	INTRODUCTION	1
1.1	General	1
1.2	Objective	2
1.3	Existing System	3
1.4	Proposed System	3
CHAPTER 2	LITERATURE SURVEY	5
2.1	Literature Review	5
2.2	Comparison and Discussion	7
2.3	Conclusion	7
CHAPTER 3	SYSTEM DESIGN	8
3.1	System Architecture	8
3.2	System Requirements	9
3.2.1	Software Requirements	9
3.2.2	Hardware Requirements	10
3.2.3	Data Requirements	10
3.2.4	Deployment and Scaling	11
3.3	System Design	11
3.3.1	Activity Diagram	12
3.3.2	Data Flow Diagram	13

CHAPTER 4	PROJECT DESCRIPTION	14
4.1	Methodologies	14
4.2	Modules	15
4.2.1	Data & Authentication Module	16
4.2.2	Ledger & Party Management Module	17
4.2.3	Transaction & Credit Tracking Module	17
4.2.4	Voice & NLP Module	18
4.2.5	Invoice, UPI & Reporting Module	18
4.2.6	UI / Dashboard Module	19
CHAPTER 5	IMPLEMENTATION AND RESULT DISCUSSION	20
5.1	Implementation Results	20
5.1.1	Implementation of Data Model & Persistence	21
5.1.2	Implementation of Voice Transaction Parsing	22
5.1.3	Implementation of Ledger Workflow & Reports	23
5.2	Discussion	24
CHAPTER 6	CONCLUSION AND FUTURE WORK	25
6.1	Conclusion	25
6.2	Future Work	25
	REFERENCES	26

LIST OF TABLES

Table No.	Title	Page No.
Table 1	Comparative Analysis	7
Table 2	Software Requirements	9
Table 3	Hardware Requirements	10
Table 4	Core Modules and Responsibilities	19
Table 5	Phase-1 Functional Verification Summary	23

LIST OF FIGURES

Figure No.	Title	Page No.
Fig. 1	Overview of Proposed System	4
Fig. 2	Architecture Diagram	8
Fig. 3	Activity Diagram	12
Fig. 4	Data Flow Diagram	13
Fig. 5	Functional Coverage Bar Graph	24

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ER	Entity-Relationship
FK	Foreign Key
GST	Goods and Services Tax
HTML	HyperText Markup Language

NLP	Natural Language Processing
ORM	Object-Relational Mapping
PDF	Portable Document Format
PK	Primary Key
PWA	Progressive Web Application
QR	Quick Response
RBAC	Role-Based Access Control
SDG	Sustainable Development Goal
SQL	Structured Query Language
STT	Speech-to-Text
TTS	Text-to-Speech
UI	User Interface
UPI	Unified Payments Interface
URI	Uniform Resource Identifier
UX	User Experience

CHAPTER 1

INTRODUCTION

1.1 General

Micro-enterprises form an important part of everyday commerce. Small shopkeepers, kirana stores, street vendors, repair units, home-based businesses, tea stalls, fruit sellers, and local retailers often operate with very limited administrative support. Their financial activity is frequent but made up of small-value transactions: cash sales, digital payments, customer credit, supplier purchases, operating expenses, and partial settlements. For many such businesses, the most practical bookkeeping method remains a handwritten notebook or informal message history.

Paper-based ledgers are familiar and inexpensive, but they create operational problems as transaction volume grows. A business owner may need to locate a customer's old credit entry, verify whether a partial payment was recorded, compare today's expenses with sales, or determine which supplier amount remains due. In a notebook these tasks require manual searching and repeated arithmetic. Pages can be damaged, entries can be written unclearly, and there is no automatic backup or consolidated financial view.

The Micro Entrepreneur Ledger App is designed as a simple digital alternative focused on the day-to-day language of small businesses rather than on complex accounting terminology. The application records parties, transactions, dues, payments, expenses, purchases, and sales in a structured ledger. Every entry contributes to an immediately recalculated balance, while searchable transaction history and summary views help the user understand where money was received, where it was spent, and which amounts are still pending.

The current Phase-1 implementation uses a modern full-stack web architecture. Next.js and React provide the interface, server-side actions process ledger operations, Prisma ORM maps application objects to SQLite records, and a secure authentication layer separates one business user's information from another. The use of SQLite keeps the prototype lightweight and easy to run during development while preserving relational constraints and queryable transaction history.

1.2 Objective

The first objective of the system is to replace unstructured paper bookkeeping with a consistent digital ledger. Every financial event should be recorded with a date, transaction type, amount, related customer or supplier when applicable, and an optional note. The system should preserve this information in a searchable form so that a user can review past activity without manually scanning multiple notebook pages.

The second objective is to make customer credit and payment tracking easy to understand. Micro-businesses frequently allow trusted customers to buy now and pay later, and partial payments can make the outstanding balance difficult to remember. The application therefore maintains party-wise ledger histories and continuously derives the amount receivable or payable. A business owner can open a customer or supplier record and see the sequence of credits, payments, and the resulting balance.

The third objective is to reduce the amount of typing and technical knowledge required for regular entry. The prototype provides clear forms with limited fields and also introduces voice-assisted transaction entry. Through the Web Speech API, spoken phrases can be converted into text, and a custom heuristic NLP layer extracts the likely party name, amount, transaction intent, and note from English, Tamil-oriented speech recognition output, or Tanglish-style commands. The parsed transaction is shown for confirmation before it is committed to the ledger.

The fourth objective is to connect record keeping with practical payment and reporting tasks. The system can generate invoice-oriented views and UPI QR codes from a standard UPI payment URI so that a due amount can be presented together with a convenient payment option. Daily and monthly summaries, visual charts, CSV export, printable HTML reports, and PDF-oriented output help the user convert raw transactions into understandable business information.

The final objective is to maintain security and future extensibility. Authentication is handled through NextAuth.js, passwords are stored using bcryptjs hashing rather than plain text, and the database model is accessed through Prisma rather than ad-hoc SQL in the interface. The design deliberately keeps the business rules independent enough that later versions can add stronger offline support, cloud synchronization, GST-compliant invoicing, and predictive analytics without replacing the core ledger model.

1.3 Existing System

The existing bookkeeping process used by many small businesses is predominantly manual and fragmented. Financial events are written in notebooks, remembered mentally, stored as chat messages, or reconstructed from payment-app histories. These sources may be individually useful, but they do not create one consistent business ledger.

- Customer dues and supplier balances are calculated manually, which increases the risk of arithmetic errors when there are many small transactions or partial payments.
- Paper records can be misplaced, damaged, or become difficult to search as the number of pages and customers increases.
- Cash transactions and digital payments are often tracked separately, so the owner must combine information from several places before estimating daily income or expenses.
- Traditional notebooks provide no automatic filtering, reminders, category-wise summaries, charts, backup, or structured invoice generation.
- Existing general-purpose accounting software can be powerful but may require more accounting terminology, configuration, or data entry than a very small vendor is willing to perform for everyday use.
- Payment applications show transfers but do not by themselves represent the complete business context: a transfer may be a sale, repayment of old credit, supplier settlement, or personal transaction.

1.4 Proposed System

The proposed Micro Entrepreneur Ledger App brings these separate tasks into one focused bookkeeping workflow. The owner creates a business profile, manages customers or suppliers, records sales, purchases, expenses, credit, and payments, and receives an updated ledger balance after every transaction. Transaction history can be searched and filtered by date, party, type, or category, while dashboard summaries present the same data in a more readable form.

For accessibility, a transaction can be entered either through a conventional form or through a voice-assisted flow. Speech recognition converts audio to text; the custom NLP parser identifies financial intent and extracts values; validation checks the required fields; and the user confirms the structured result before storage. This keeps the convenience of speech while ensuring that a financial record is not silently created from an uncertain interpretation.

The proposal also connects bookkeeping with payment and reporting support. Invoice data can be transformed into a printable bill, and a UPI URI can be encoded as a QR code for the relevant amount and payee information. Reports can be viewed on-screen, exported as CSV, or printed to PDF through standard browser printing. Recharts-based visualizations provide trends without requiring the user to calculate totals manually.

The architecture is intentionally lightweight for the Phase-1 prototype. The application runs through Next.js on Node.js, with Prisma ORM and SQLite providing the persistence layer. This differs from the earliest concept document, which emphasized full offline-first cloud synchronization as an initial capability. In the present implementation, reliable ledger operations, voice-assisted entry, reports, and payment support are prioritized first; deeper offline packaging and multi-device cloud synchronization are treated as deployment extensions after core workflow validation.

The project is aligned with SDG 8 because its target users are micro-entrepreneurs and small businesses. UN Target 8.3 explicitly emphasizes policies that support entrepreneurship and the formalization and growth of micro-, small-, and medium-sized enterprises, including through access to financial services. A practical digital ledger does not itself provide credit, but it can improve the quality and traceability of business records on which everyday financial decisions depend.

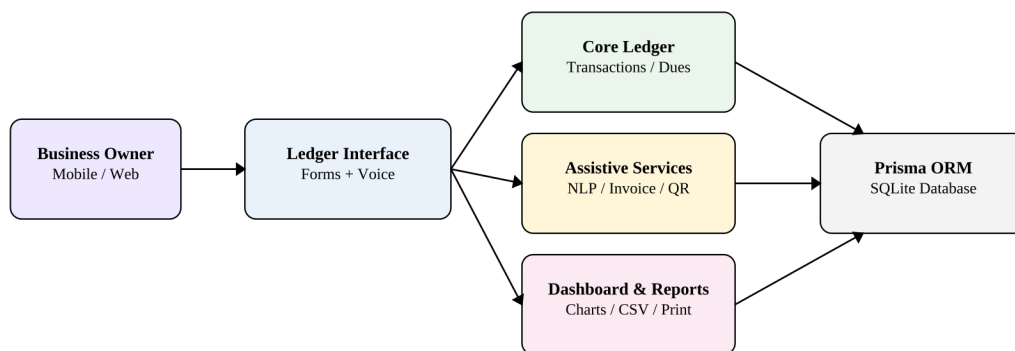


Fig. 1 - Overview of Proposed System

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Review

Digital financial technology has been widely discussed as an enabler of more inclusive and efficient financial services. The World Bank's *Fintech and the Future of Finance* notes that digitization can improve efficiency and expand access while also creating new requirements around consumer protection, data, and regulation [1]. For a micro-entrepreneur, the relevant lesson is that digital finance is most useful when the supporting information is structured. A payment record alone says that money moved; a ledger adds the business meaning of why it moved, who it relates to, and whether an obligation remains.

Human-computer interaction research is particularly relevant because the target users of this project may be experienced business operators without being experienced software users. Medhi et al. studied novice and low-literacy users across multiple countries and found that conventional text-heavy interfaces can become significant barriers, while graphical and speech-based interaction can improve usability for appropriate tasks [2]. Their earlier CHI study comparing text, spoken-dialog, and rich multimedia mobile money interfaces similarly reported strong preference for non-text approaches among non-literate and semi-literate participants [3]. These findings support the project's use of large, obvious actions, limited form fields, and an optional voice path rather than relying on dense accounting screens.

More recent work by Srivastava et al. synthesized actionable guidelines for smartphone applications inclusive of low-literate users [4]. The study emphasizes reducing reading burden, using recognizable visual cues, minimizing deep navigation, and designing interactions around users' existing mental models. For the Micro Entrepreneur Ledger App, this translates into familiar labels such as sale, expense, due, and payment; clear party balances; direct navigation to transaction entry; and visible confirmation before data is stored.

A second technical theme is resilience to inconsistent connectivity. Kleppmann et al.'s local-first software principles argue that applications should preserve useful local operation, low latency, and user control rather than making every interaction dependent on a remote service [5]. The project's earliest concept adopted this idea strongly through an offline-first design. The Phase-1 prototype uses local SQLite persistence on the application side as a practical starting point, while fuller offline synchronization is reserved for a subsequent deployment stage.

Payment integration is another relevant area. India's Unified Payments Interface has become a dominant mechanism for everyday account-to-account digital payment, and NPCI publishes transaction statistics at national scale [6]. UPI's URI-based payment mechanism and QR presentation make it possible for a ledger application to generate a payment request without becoming a payment processor itself. The ledger app therefore treats UPI QR generation as a convenience attached to an invoice or outstanding amount, while the actual payment continues to be handled by the user's UPI application and banking infrastructure.

The implementation also relies on established web-platform technologies rather than creating proprietary infrastructure. Next.js provides a React-based full-stack application framework; in version 14, Server Actions are stable for server-side mutations and Route Handlers support HTTP request handling [7][8]. Prisma ORM provides typed database access and supports SQLite as a lightweight file-based relational database suitable for prototypes and smaller applications [9]. The browser Web Speech API exposes speech recognition and synthesis capabilities that can be used to build voice-assisted interfaces, although support and behavior depend on the browser and recognition service [10].

Security literature and guidance are equally relevant because financial records are sensitive even when the application is not a bank. OWASP recommends dedicated password hashing algorithms and appropriate work factors rather than reversible storage; the prototype uses bcryptjs for password hashing within its authentication flow [11]. The design also keeps transaction mutation on the server side, validates parsed voice data before persistence, and avoids treating the speech transcript itself as an authoritative financial record until the user confirms the structured fields.

2.2 Comparison and Discussion

The reviewed work can be grouped into four complementary directions: digital financial services and bookkeeping, inclusive/low-literacy interface design, offline or local-first data architecture, and payment integration. Each direction contributes an important piece, but none alone represents the complete problem faced by a small vendor who needs to record informal credit, accept mixed cash and digital payments, and understand outstanding dues with minimal interaction overhead.

Table 1 - Comparative Analysis

Evaluation Dimension	Digital Bookkeeping / Fintech	Inclusive UI / Voice	Local-First / Offline	UPI / Payment Integration
Core focus	Structured financial records and access	Reduce literacy and interaction barriers	Usable data without constant network	Fast account-to-account payment
Primary advantage	Improves traceability and summaries	Lower typing and reading burden	Continuity, latency, and user control	Convenient merchant payment request
Limitation when used alone	May remain complex for novice users	Does not define ledger semantics	Synchronization adds conflict complexity	Payment history is not full bookkeeping

Gap This Project Targets	Simple party-wise ledger plus daily/monthly views	Voice entry with confirmation and familiar	Lightweight local relational persistence with future sync	Invoice-linked UPI QR while ledger remains source of context
--------------------------	---	--	---	--

The comparison shows that the project is not attempting to replace full accounting packages or payment networks. Its contribution is the combination of a small-business ledger with an interface designed around low-friction transaction entry. The application retains structured relational data while presenting it through familiar business actions; it offers speech as an optional input method rather than as an uncontrolled automation path; and it attaches invoice and UPI support to the ledger rather than treating payment receipts as the ledger itself.

This combination also clarifies the project's technical boundaries. Voice recognition can fail because of pronunciation, ambient noise, browser behavior, or mixed-language speech; therefore confirmation is required. SQLite keeps the prototype simple but is not the final choice for a large multi-tenant cloud service. UPI QR generation creates a standards-based payment request but does not independently verify settlement. These limitations are manageable because each external capability is isolated from the core transaction model.

2.3 Conclusion

The literature survey establishes a practical foundation for the Micro Entrepreneur Ledger App. Research on low-literacy interfaces supports graphical simplicity and optional spoken interaction; local-first principles support continued attention to data availability and user control; digital finance research motivates structured financial records; and UPI demonstrates the value of interoperable payment mechanisms. The resulting design goal is therefore not maximum feature density, but an understandable ledger that can capture common business events quickly, preserve them reliably, and convert them into balances, invoices, and summaries without forcing the user to learn formal accounting procedures.

The survey also supports the staged development strategy adopted in this phase. Core correctness of transaction storage and balance computation comes first. Assistive features such as heuristic voice parsing are added around that core with explicit confirmation, while cloud synchronization, GST compliance, and predictive functions remain separate future extensions. This keeps the prototype demonstrable and auditable while retaining a clear path toward the broader mobile-first vision.

CHAPTER 3

SYSTEM DESIGN

3.1 System Architecture

The Micro Entrepreneur Ledger App follows a three-tier full-stack web architecture with an additional service layer for assistive functions. The **Presentation Layer** is implemented with Next.js 14, React 18, Tailwind CSS, Framer Motion, and Lucide React. It renders the dashboard, party ledgers, transaction forms, voice-entry controls, reports, charts, invoices, and authentication screens in a responsive layout.

The **Application/API Layer** runs on Node.js through Next.js Server Actions and Route Handlers. Server Actions handle trusted mutations initiated by the interface, such as creating a customer or recording a transaction, while Route Handlers are used when a conventional request/response endpoint is appropriate. Validation is performed before values reach the persistence layer.

The **Domain Services Layer** contains the bookkeeping rules and auxiliary services: party balance calculation, transaction classification, custom heuristic NLP for voice commands, report aggregation, invoice preparation, UPI URI construction, and export formatting. The **Data Persistence Layer** uses Prisma ORM over SQLite. Prisma models express relationships between users, businesses, parties, transactions, invoices, and related records while giving the server type-safe database access.

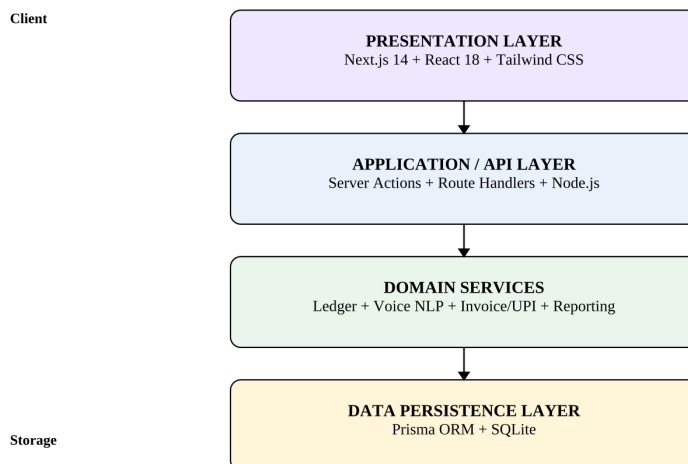


Fig. 2 - Architecture Diagram

Authentication is treated as a cross-cutting concern. NextAuth.js maintains authenticated sessions, bcryptjs is used for one-way password hashing, and protected server actions operate in the context of the logged-in user. The design does not expose raw database operations to the client. Voice recognition is intentionally client-side because the Web Speech API is a browser capability; only the parsed and confirmed transaction data is submitted to the server.

3.2 System Requirements

The Phase-1 prototype is designed to run on common development hardware and a modern Chromium-based browser. No GPU or specialized machine-learning runtime is required because the language component is a heuristic parser rather than a trained deep-learning model.

3.2.1 Software Requirements

Table 2 - Software Requirements

Software Component	Technology / Tool	Version / Specification
Operating System	Windows / Linux / macOS	Windows 10/11 or equivalent
Runtime	Node.js	LTS runtime compatible with Next.js 14
Frontend	Next.js + React	Next.js 14, React 18
Styling / UI	Tailwind CSS, Framer Motion, Lucide	Responsive interface and icons
Server Logic	Server Actions + Route Handlers	Next.js App Router
ORM / Database	Prisma ORM + SQLite	Relational local database
Authentication	NextAuth.js + bcryptjs	Session auth and password hashing
Voice	Web Speech API	Browser speech recognition / synthesis
NLP	Custom heuristic parser	English, Tamil-oriented, Tenglish patterns
Charts	Recharts	Financial summaries and trends
Invoice / QR	qrcode.react + UPI URI	Payment QR rendering
Reporting	CSV + HTML5 print + PDF output	Export and printable reports

3.2.2 Hardware Requirements

Table 3 - Hardware Requirements

Hardware Category	Specification	Purpose in System
Processor (CPU)	Dual-core 64-bit or better	Runs Node.js server and browser client
System Memory	Minimum 4 GB; 8 GB recommended	Development server, browser, local database
Storage	Minimum 2 GB free for prototype	Source code, SQLite database, exports
Microphone	Built-in or external microphone	Voice-assisted transaction entry
Network	Required for deployment / speech service as application	Application install, hosted access, browser speech recognition

3.2.3 Data Requirements

The application does not require a machine-learning training dataset. Its core data is operational bookkeeping data generated by the user during normal business activity. A minimal deployment requires records for the user and business profile, parties such as customers or suppliers, and dated financial transactions. Optional invoice and report records are derived from the same ledger data rather than maintained as a separate accounting truth.

A transaction record contains enough information to reconstruct the ledger: owner/business scope, party when applicable, transaction type, amount, date and time, category or description, payment mode, and an optional note. Credit-related entries affect the party's outstanding balance, while cash sales, purchases, and expenses affect summary totals. The system uses relational identifiers so that a transaction cannot accidentally appear under an unrelated business account.

For voice input, the raw transcript may be retained temporarily for user confirmation, but the authoritative record is the validated structured transaction. This separation prevents changes in speech-recognition output from corrupting accounting data and allows the user to correct a name or amount before saving.

3.2.4 Deployment and Scaling

The present prototype is appropriate for a single-machine or small hosted demonstration. SQLite offers low setup overhead and is well suited to development, prototypes, and smaller applications. For a larger multi-user deployment, the Prisma data model can be migrated to a server database such as PostgreSQL, while the Next.js application can be deployed behind HTTPS with centralized backups and production session management.

The original project concept also included automatic cloud synchronization and stronger offline-first operation. Those capabilities require explicit conflict-resolution and device synchronization design, particularly when the same business record can be changed on multiple devices. They are therefore treated as a controlled future extension rather than simulated in the Phase-1 results.

The voice component requires special deployment consideration because browser speech-recognition support varies by platform. The application should always provide a manual form as a complete fallback. Likewise, UPI QR generation is only a payment request: a future production version should reconcile confirmed payments using an authorized payment or banking integration rather than assuming that displaying a QR code proves settlement.

3.3 System Design

The system design converts the architecture into a predictable business workflow. An authenticated owner operates within a selected business. Parties are created once and reused across transactions. Every ledger mutation passes through validation before Prisma writes it to SQLite. Summary calculations read from the same transaction records, ensuring that the dashboard, customer ledger, report, and invoice are different views over one consistent source of financial truth.

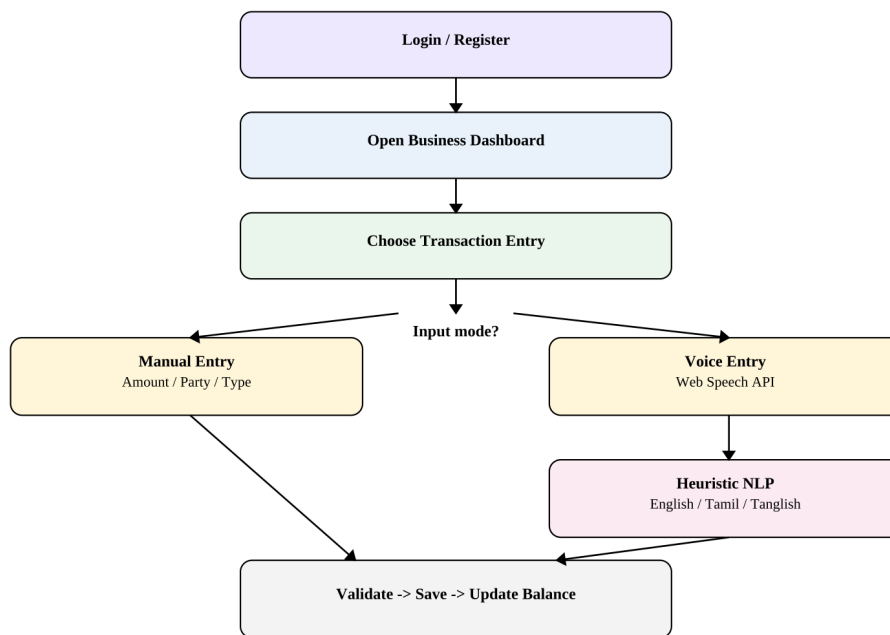
For credit transactions, the sign of the balance is controlled by transaction semantics rather than by arbitrary user-entered positive and negative numbers. This simplifies the interface and reduces accidental reversals. A customer's purchase on credit increases the amount receivable; a customer payment decreases it. Supplier purchases and settlements follow the corresponding payable logic. Deletion or correction is recorded through controlled actions so derived totals remain consistent.

Voice input is inserted before validation, not after storage. The parser first attempts to identify the transaction intent and amount from the transcript, then matches a party or leaves the field unresolved. Only after the user sees and confirms the interpreted values does the normal transaction creation path execute. This design keeps the ledger rules identical whether input begins as typing or speech.

3.3.1 Activity Diagram

The activity flow begins when a registered user signs in and opens the business dashboard. The owner chooses a direct transaction form or voice-assisted entry. Manual entry proceeds directly to validation. Voice entry first captures speech, converts it to a transcript, and applies the heuristic NLP parser. In both paths, the resulting structured fields are checked before they are saved.

After a valid transaction is stored, the party ledger and dashboard summaries are recalculated from persisted records. The user can then continue entering transactions or generate an invoice, UPI QR, or report. Any failure before persistence returns the user to a correction state; the database is not updated with a partially interpreted transaction.



Optional: Invoice / UPI QR / Export Report

Fig. 3 - Activity Diagram

3.3.2 Data Flow Diagram

At Level 0, the Business Owner is the primary actor supplying authentication data, party details, transactions, and report requests to the Micro Entrepreneur Ledger App. Customers and suppliers do not require their own application accounts in the Phase-1 design; their names, balances, invoices, and payment information are managed by the business owner. An invoice or UPI request may be shared outward, but ledger mutation remains controlled by the authenticated owner.

At Level 1, the system decomposes into authentication, ledger/party management, transaction processing, voice NLP, invoice/UPI generation, and reports/analytics. These processes exchange structured records through the SQLite data store. Voice NLP never writes directly to the database: it outputs candidate transaction fields to the transaction process. Reports read ledger data and aggregate it without changing the underlying financial records.

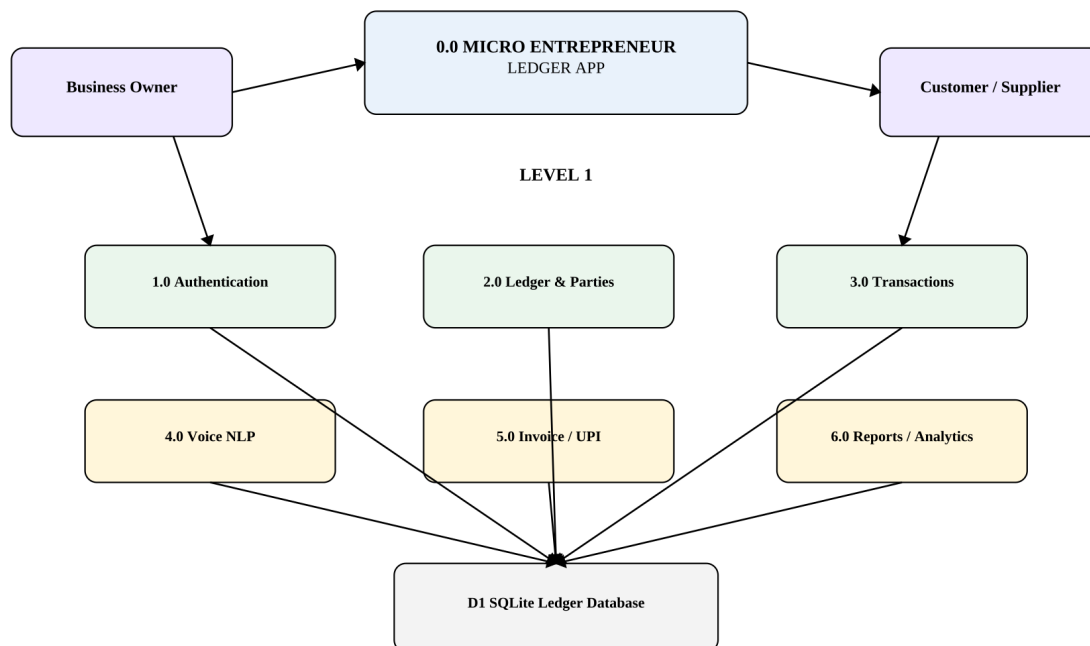


Fig. 4 - Data Flow Diagram (Level 0 Context and Level 1)

CHAPTER 4

PROJECT DESCRIPTION

4.1 Methodologies

The project follows an iterative full-stack development methodology in which the accounting model is built first and assistive features are added around it. The objective is to keep the ledger rules deterministic and easy to verify while using heuristic language processing only for convenience of input.

- **Relational Ledger Modelling:** Users, businesses, parties, transactions, invoices, and supporting categories are represented as related entities. Prisma manages the schema and database access while SQLite provides lightweight persistence for the Phase-1 prototype.
- **Server-Side Transaction Processing:** Financial mutations are performed through Next.js Server Actions or controlled Route Handlers. Input is validated on the server before database writes, preventing client-side state from becoming the only record of a financial event.
- **Deterministic Balance Aggregation:** Customer and supplier balances are calculated from transaction semantics. Sales, credit, receipts, purchases, payments, and expenses contribute to defined summary categories rather than using a black-box prediction model.
- **Heuristic Voice Parsing:** Web Speech API output is normalized and scanned for amount expressions, transaction keywords, payment intent, and likely party references. English, Tamil-oriented vocabulary, and Tanglish patterns are supported through explicit rules and aliases. The user confirms the result before saving.
- **Invoice and UPI Generation:** Invoice data is composed from business, party, and transaction information. A standard UPI payment URI is assembled from payee and amount information and encoded into a QR component using `qrcode.react`.
- **Reporting and Visualization:** Database queries aggregate daily and monthly totals for sales, expenses, dues, and payments. Recharts visualizes the summaries, while CSV export and browser printing provide portable records.

This mixed methodology is deliberate. Core accounting behavior must be predictable; speech interpretation can be approximate but must never bypass confirmation; and payment QR generation must remain separate from settlement verification. Each method is therefore selected for the reliability required by its part of the workflow.

4.2 Modules

The application is divided into six cohesive modules that cover the lifecycle from secure login to recording a transaction and producing useful business summaries.

Module 1: Data & Authentication Module. This module creates the secure application boundary. It manages users, business ownership, authenticated sessions, password hashing, and Prisma access to the SQLite database. It also defines the relational models and ensures that records are scoped to the correct business.

Module 2: Ledger & Party Management Module. This module manages customers, suppliers, and their ledger histories. It supports adding and editing party information, searching parties, viewing chronological transactions, and deriving receivable or payable balances from the associated records.

Module 3: Transaction & Credit Tracking Module. This module records sales, purchases, expenses, customer credit, receipts, and supplier payments. It validates amount and type, connects transactions to parties or categories, maintains transaction history, and provides filtering or correction actions.

Module 4: Voice & NLP Module. The browser captures speech through the Web Speech API. A custom parser normalizes the transcript and extracts likely intent, amount, party, and note. The module supports common English and mixed-language expressions and always routes the parsed result through user confirmation.

Module 5: Invoice, UPI & Reporting Module. This module builds invoice views, creates UPI payment QR codes, calculates business summaries, and exports records through CSV or printable/PDF-oriented reports. Recharts visualizations are produced from the same ledger aggregates.

Module 6: UI / Dashboard Module. This module provides the mobile-first interaction layer: dashboard cards, prominent transaction actions, party search, ledger views, charts, invoice presentation, and responsive navigation. Tailwind CSS, Framer Motion, and Lucide React support the visual implementation.

4.2.1 Data & Authentication Module

The Data & Authentication Module is the foundation of the platform. A registered user authenticates through NextAuth.js, and password credentials are stored as bcryptjs hashes rather than as readable values. Once authenticated, server-side code resolves the active user and restricts business data operations to records owned by that account. This prevents a client request from selecting an unrelated user's ledger merely by changing an identifier in the browser.

Prisma ORM acts as the only normal path to relational storage. Its schema defines primary keys, foreign-key relationships, timestamps, and application-level enums or categories used by the ledger. SQLite stores the Phase-1 data in a compact file-based database that is easy to inspect and back up during development. Because the application accesses data through Prisma models, the same domain model can later be migrated to a production database with less disruption than embedding raw SQLite syntax throughout the UI.

Typical records in this module include User, Business, Party, Transaction, Invoice, and supporting configuration. A Business belongs to a User; Parties and Transactions belong to a Business; and a Transaction may optionally reference a Party. This ownership chain is important because financial summaries and searches must never mix data from different businesses.

4.2.2 Ledger & Party Management Module

The Ledger & Party Management Module represents the people and organizations with whom the micro-entrepreneur regularly transacts. A party may be a customer who owes money, a supplier to whom money is owed, or a reusable contact associated with immediate transactions. The module stores simple identity and contact information while keeping the financial history in transaction records rather than in a manually edited balance field.

When the owner opens a party ledger, the application retrieves the relevant transaction sequence and calculates the running or current balance according to transaction type. This makes the balance reproducible: if an incorrect entry is corrected, the displayed total changes automatically because it is derived from the updated history. Search and date filters reduce the effort required to locate older entries.

The design intentionally avoids requiring the user to understand formal debit and credit notation. Interface labels describe the business action in familiar language, while the server maps those actions to the correct balance effect. This reduces cognitive load without sacrificing consistency in the stored data.

4.2.3 Transaction & Credit Tracking Module

The Transaction Module handles the most frequent operation in the application. A transaction includes an amount, business date, action type, optional party, optional category or payment mode, and a note. The server verifies that required fields exist and that the amount is valid before persistence. Financial actions are represented explicitly rather than inferred later from free text.

Customer credit and repayment are treated as linked ledger events. Creating a credit sale increases the customer's outstanding amount; recording a receipt reduces it. Supplier purchases and settlements are handled in the corresponding payable direction. General expenses such as transport, rent, or supplies can be recorded without attaching them to a customer, allowing the dashboard to separate revenue-oriented and expense-oriented totals.

Transaction history provides chronological evidence of how a balance was produced. Filtering by period or transaction type supports daily review and month-end summaries. Because the underlying records remain itemized, the application can generate reports without storing a second copy of every total.

4.2.4 Voice & NLP Module

The Voice & NLP Module provides an alternative entry path for users who find typing slower than speaking. The browser begins a speech-recognition session and receives a text transcript from the Web Speech API. The application normalizes case and punctuation, maps common numeric expressions, and applies a rule set to identify words associated with sale, expense, credit, received payment, supplier payment, and other supported intents.

The parser then searches for the amount and possible party reference. English commands such as 'received 500 from Kumar' can be interpreted directly, while Tamil-oriented or Tenglish vocabulary is supported through aliases and keyword patterns. Because mixed-language recognition quality varies by browser and accent, the module is intentionally heuristic rather than presented as perfect language understanding.

The most important control is confirmation. Parsed values are loaded into the same structured form used by manual entry, where the user can correct any field. Only the confirmed form calls the normal transaction action. This means a recognition mistake becomes an editable suggestion rather than an irreversible accounting error.

4.2.5 Invoice, UPI & Reporting Module

The Invoice, UPI & Reporting Module converts ledger information into formats that can be shared or reviewed. An invoice combines business identity, party details, one or more line descriptions or transaction references, total amount, and payment information. The current prototype focuses on practical bills and receipts rather than claiming complete statutory GST compliance.

For UPI support, the application constructs a payment URI containing the payee address and relevant amount or note, then renders it through `qrcode.react`. Scanning the QR opens a compatible UPI application with the payment details pre-filled. The ledger app does not claim that a QR has been paid merely because it was displayed; payment verification remains a separate concern for future payment-provider integration.

Reports are calculated from persisted transactions and can be filtered by date or category. The dashboard uses `Recharts` for visual summaries, while CSV export preserves row-level data for spreadsheet use. HTML5 print layouts support hard copy and browser-based PDF generation. This allows a micro-entrepreneur to move from individual entries to a daily or monthly view without maintaining a second notebook.

4.2.6 UI / Dashboard Module

The UI module is designed around short, frequent tasks. The dashboard surfaces key totals and recent activity rather than exposing every setting at once. Prominent actions allow the owner to add a sale, expense, credit, or payment, while the parties area gives direct access to customer and supplier ledgers. Responsive Tailwind CSS layouts allow the same application to remain usable on a phone-sized browser and on a larger desktop screen.

`Framer Motion` is used for lightweight interaction feedback and `Lucide React` provides consistent icons. The interface avoids relying on color alone to communicate financial meaning; textual labels, amounts, and direction indicators accompany visual cues. Error messages are placed close to the affected input so the user can correct a transaction before submission.

The voice entry control is exposed as an optional shortcut rather than the only way to add data. Similarly, charts supplement numeric totals rather than hiding them. These choices follow the broader accessibility principle that assistive features should reduce effort without making the interface unusable when a browser feature, microphone, or network-dependent recognition service is unavailable.

4.3 Core Module Responsibilities

Table 4 summarizes the six primary modules and the technology or rule set that drives each one. The system deliberately distinguishes deterministic financial logic from assistive features. Balance calculation, authentication, and database persistence are not delegated to probabilistic interpretation; voice NLP only prepares candidate values for the normal validated transaction path.

Table 4 - Core Modules and Responsibilities

#	Module	Primary Method / Technology	Type
1	Data & Authentication	NextAuth.js, bcryptjs, Prisma, SQLite	Security / persistence
2	Ledger & Party Management	Relational party ledger + derived balances	Deterministic business logic
3	Transaction & Credit Tracking	Validated server mutations and aggregation	Deterministic business logic
4	Voice & NLP	Web Speech API + heuristic parser	Assistive / rule-based NLP
5	Invoice, UPI & Reporting	UPI URI, QR rendering, CSV/print/PDF	Document / payment support
6	UI / Dashboard	Next.js, React, Tailwind, Recharts	Presentation / analytics

This separation is useful during testing and viva explanation. Not every feature described as 'smart' needs a machine-learning model. The voice parser is intentionally rule-based because the supported financial command set is narrow and because every interpretation is confirmed. The ledger itself remains transparent: totals are reproducible from stored transactions, which is more important for financial trust than hiding the result behind a predictive model.

The module boundaries also create a clear future path. A later machine-learning classifier could replace or supplement heuristic intent detection, a cloud database could replace local SQLite, and a payment reconciliation service could augment UPI QR generation. None of those changes requires rewriting the basic party and transaction semantics.

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 Implementation Results

The Phase-1 prototype brings the primary ledger functions into one Next.js application. Authenticated users can manage business records, create customers or suppliers, add financial transactions, review party histories, calculate outstanding amounts, and view aggregated financial summaries. The same server-side persistence layer is used by manual entry and voice-assisted entry so there is no separate 'voice database' or duplicate transaction source.

The interface was implemented with React components styled by Tailwind CSS and supported by Framer Motion and Lucide icons. Server Actions perform common mutations directly from the App Router workflow, while Route Handlers are available for API-style operations. Prisma Client provides database access to SQLite, ensuring that related records can be queried consistently for party ledgers, recent transactions, invoices, and reports.

Voice-assisted entry was integrated as a user-interface feature rather than a background automation. The browser produces a transcript; the parser generates candidate fields; the user confirms them; and only then does the normal create-transaction operation run. This implementation avoids giving an uncertain speech result authority over stored financial data.

5.1.1 Implementation of Data Model & Persistence

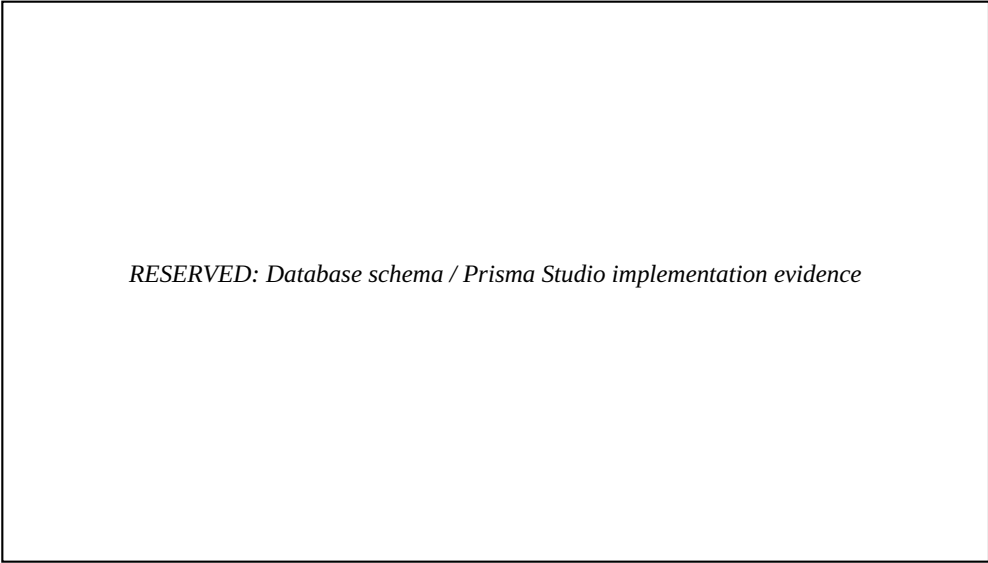
The relational model is centered on ownership and transaction history. User and Business records establish the scope of data. Parties represent customers or suppliers without storing a manually editable authoritative balance. Transactions reference the active business and, when applicable, a party. This design makes the ledger auditable because a balance can always be recalculated from the underlying transaction sequence.

Prisma migrations define the schema during development, and SQLite stores the prototype database. Common operations include creating a party, inserting a transaction, filtering by business and date, summing transaction groups for dashboard cards, and retrieving all transactions for a specific party in chronological order.

Server-side validation prevents empty or invalid amounts from becoming ledger entries. Ownership checks are performed with the authenticated user context before the database mutation. If a transaction is corrected or removed, reports and party balances reflect the revised record set the next time they are queried. The prototype therefore avoids keeping independent manually synchronized totals that can drift away from the source transactions.

The use of SQLite is appropriate for this development stage because it is simple to inspect and requires no separate database server. It should not be interpreted as the final scaling choice. A production cloud deployment with many concurrent businesses would normally move to a managed relational database, stronger backup procedures, and explicit monitoring.

The following reserved area is intended for a final screenshot of the Prisma schema or database inspection after the team selects the exact evidence to include in the submitted report.



RESERVED: Database schema / Prisma Studio implementation evidence

Implementation evidence placeholder - to be replaced with final project screenshot

5.1.2 Implementation of Voice Transaction Parsing

The voice implementation begins with a microphone action on the transaction screen. Browser speech recognition returns a transcript that is normalized before parsing. The heuristic logic searches for known financial verbs and aliases, amount patterns, and recognizable party names. Examples of supported intent classes include sale, expense, credit given, payment received, purchase, and supplier payment.

The parser is designed to be explainable. Instead of producing an opaque confidence score, it exposes the fields it found and leaves unresolved values blank. For example, if an amount is detected but the party is not recognized, the form can still present the amount while requiring the user to choose the party manually. English and Tanglish-style phrases can share the same normalized intent dictionary, and Tamil-oriented aliases can be added without retraining a model.

Speech recognition itself remains the main source of uncertainty. Microphone quality, ambient noise, pronunciation, browser implementation, and language settings can alter the transcript. The implementation therefore treats the voice layer as a productivity aid and retains the standard form as a complete fallback.

No accuracy percentage is reported in this Phase-1 document because the team has not supplied a controlled labelled command dataset and test protocol. The final report may add command-level precision or correction-rate measurements after representative test phrases are collected under defined conditions.

5.1.3 Implementation of Ledger Workflow & Reports

Once a confirmed transaction is submitted, the server stores it through Prisma and the interface refreshes the relevant ledger and dashboard values. The same records drive invoices, charts, and exports. This implementation reduces duplication: a daily report does not maintain a separate total that can disagree with the customer ledger.

Table 5 - Phase-1 Functional Verification Summary

Functional Area	Verification Basis	Phase-1 Result
Authentication	Session-protected application flow	Implemented in prototype
Ledger CRUD	Create/read/update transaction and party records	Implemented in prototype
Balance derivation	Aggregate persisted transaction semantics	Implemented in prototype
Voice entry	Transcript -> parse -> confirm -> save flow	Prototype integrated; dataset metric reserved
Invoice / UPI QR	Invoice data + UPI URI encoded as QR	Prototype integrated
Reports / exports	Dashboard aggregation, CSV, print/PDF flow	Prototype integrated

The implemented reporting flow groups records by time period and transaction class. Recharts visualizations are generated from aggregate data, while CSV export preserves tabular details for external analysis. Printable invoice and report layouts use browser print behavior so the owner can keep a PDF or physical copy without requiring a separate desktop reporting application.

Figure 5 summarizes the number of core functional flows documented in each module of the current prototype. The counts are a structural coverage view, not a performance or accuracy claim. They are included to show that the application is distributed across security, ledger, transaction, voice, invoice/payment, and reporting concerns rather than concentrating all logic in a single page.

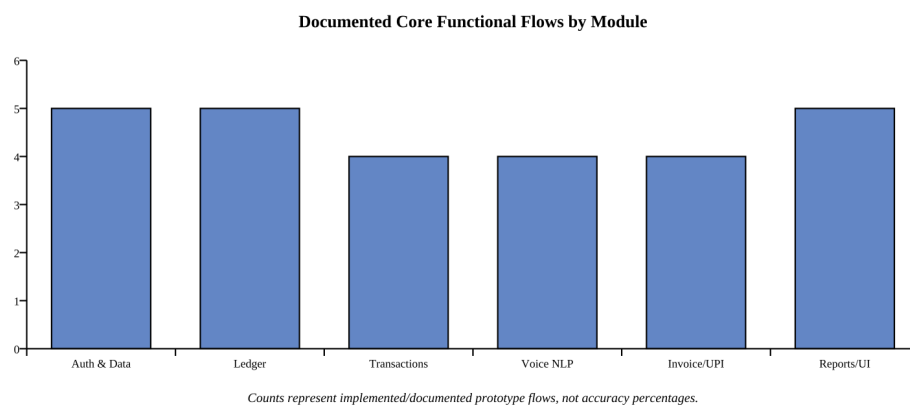


Fig. 5 - Functional Coverage Bar Graph

The prototype demonstrates the intended end-to-end sequence: authenticate, select a business, create or locate a party, record a transaction through manual or voice-assisted entry, persist the record, recalculate the relevant balance, and expose the result through history, dashboard, invoice, or export views. This sequence is more important at Phase 1 than synthetic throughput numbers because the project is targeted at a small-business workflow, not a high-frequency trading system.

Reserved quantitative measurements for the next review include voice-command correction rate, average time to record a manual versus voice transaction, task completion during user testing, and report generation latency on the target deployment device. Those values should be inserted only after the team executes a defined test protocol.

5.2 Discussion

The implementation illustrates a useful separation between financial correctness and interaction convenience. The ledger uses deterministic rules and relational storage because balances must be reproducible. Voice NLP is heuristic because its purpose is to reduce typing for a limited command vocabulary, and the risk of misinterpretation is controlled by confirmation. This is preferable to treating a language model or speech transcript as the final financial authority.

The use of Next.js for both interface and server logic simplifies the Phase-1 architecture. React components, Server Actions, Route Handlers, and authentication remain in one application codebase, while Prisma provides a clear boundary around persistence. For a student prototype this reduces infrastructure overhead and makes the complete flow easier to demonstrate. The trade-off is that a larger production deployment would need more explicit observability, stronger database infrastructure, backup and recovery procedures, and possibly background processing for synchronization.

SQLite is a strength for prototyping and a limitation for scaling. It provides a real relational database with very little setup, but it is not intended here as evidence that a multi-tenant production financial platform should remain file-based. The design is therefore evaluated on correctness and clarity of the ledger workflow rather than on unsupported claims of large-scale concurrency.

The voice feature is similarly bounded. Research supports the value of speech and graphical interaction for novice and low-literacy users, but browser speech recognition does not guarantee identical behavior across accents and devices. The manual form remains essential, and any future claim of multilingual accuracy should be based on a labelled test set that includes representative speakers and realistic shop environments.

The UPI QR module improves convenience without turning the application into a payment gateway. It prepares a payment request, but settlement confirmation is not inferred from the QR alone. Future reconciliation should rely on authorized payment-provider or bank-side confirmation. This boundary is important because bookkeeping must distinguish a requested payment from money actually received.

Overall, the Phase-1 system demonstrates that a micro-business ledger can remain technically simple while still combining useful modern interaction features. The next stage should emphasize user testing, offline synchronization design, production database migration, and objective measurements rather than adding features merely to increase complexity.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The Micro Entrepreneur Ledger App was developed to provide small vendors and micro-entrepreneurs with a simpler alternative to paper-based bookkeeping. The Phase-1 prototype establishes the core digital ledger: authenticated business access, customer and supplier management, structured transaction entry, credit and payment tracking, derived balances, searchable history, dashboard summaries, invoice support, UPI QR generation, and export-oriented reporting.

The application uses Next.js 14 and React 18 for the interface, Next.js server capabilities for business logic, Prisma ORM with SQLite for relational persistence, and NextAuth.js with bcryptjs for authentication. A Web Speech API based input path and a custom heuristic NLP parser provide optional voice-assisted entry in English and mixed-language usage, but the interpreted values are always presented for confirmation before they become financial records.

The project aligns with SDG 8 by focusing on digital tools for entrepreneurship and the operational growth of micro and small businesses. Its contribution is not a new payment network or a complex accounting engine; it is the integration of accessible day-to-day bookkeeping, credit visibility, assistive entry, payment request support, and understandable summaries in one lightweight workflow.

6.2 Future Work

- Complete the original offline-first vision with a PWA/local cache and conflict-safe cloud synchronization across devices.
- Migrate production storage from local SQLite to a managed relational database such as PostgreSQL with automated backup and recovery.
- Conduct structured usability testing with micro-entrepreneurs and measure task completion time, correction rate, and learnability for manual and voice entry.
- Build a labelled multilingual voice-command dataset before reporting accuracy metrics or introducing a trained intent-classification model.
- Add authorized payment reconciliation so UPI payments can be matched to ledger entries after confirmed settlement.
- Extend invoice support toward GST-compliant formats where legally appropriate, with configurable tax details and numbering.
- Add optional reminder delivery, richer multilingual UI localization, and role-based access for staff or family members assisting the business.
- Explore predictive cash-flow or sales insights only after sufficient real transaction history is available and after privacy and explainability requirements are defined.

REFERENCES

- [1] World Bank, *Fintech and the Future of Finance: Market and Policy Implications*, World Bank Group, 2022-2023 publication series.
- [2] I. Medhi, S. Patnaik, E. Brunskill, S. N. N. Gautama, W. Thies, and K. Toyama, “Designing Mobile Interfaces for Novice and Low-Literacy Users,” *ACM Transactions on Computer-Human Interaction*, vol. 18, no. 1, Article 2, 2011, doi: 10.1145/1959022.1959024.
- [3] I. Medhi, S. N. N. Gautama, and K. Toyama, “A Comparison of Mobile Money-Transfer UIs for Non-Literate and Semi-Literate Users,” in *Proc. CHI 2009*, 2009, doi: 10.1145/1518701.1518970.
- [4] A. Srivastava, S. Kapania, A. Tuli, and P. Singh, “Actionable UI Design Guidelines for Smartphone Applications Inclusive of Low-Literate Users,” *Proc. ACM Human-Computer Interaction*, vol. 5, CSCW1, 2021, doi: 10.1145/3449210.
- [5] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-First Software: You Own Your Data, in Spite of the Cloud,” in *Onward! 2019*, 2019, doi: 10.1145/3359591.3359737.
- [6] National Payments Corporation of India (NPCI), “Unified Payments Interface (UPI) Product Statistics,” official UPI statistics portal, accessed 2026.
- [7] Vercel, “Next.js 14 Documentation: Introduction,” Next.js documentation, version 14.
- [8] Vercel, “Next.js Server Actions and Route Handlers,” Next.js App Router documentation.
- [9] Prisma, “Prisma ORM with SQLite,” Prisma Documentation, SQLite connector and quick-start guides.
- [10] Mozilla Developer Network, “Web Speech API,” MDN Web Docs.
- [11] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series.
- [12] United Nations Department of Economic and Social Affairs, “Sustainable Development Goal 8: Decent Work and Economic Growth,” including Target 8.3 on entrepreneurship and the growth of micro-, small- and medium-sized enterprises.

- [13] React Team, “React Documentation,” React developer documentation, React 18 concepts and component model.
- [14] Tailwind Labs, “Tailwind CSS Documentation,” utility-first CSS framework documentation.
- [15] Recharts, “Recharts Documentation,” composable charting library for React.
- [16] NextAuth.js / Auth.js Project, “Authentication Documentation,” session and credential authentication guidance.
- [17] SQLite Consortium, “SQLite Documentation,” transactional relational database documentation.
- [18] qrcode.react contributors, “qrcode.react,” React QR-code rendering library documentation.
- [19] Manoj Kumar S, Mohamed Ikram KHD, Kamalesh L, and Mithun AR, “Micro-Entrepreneur Ledger App: Zeroth Review / Project Phase-I Materials,” Department of Computer Science and Engineering, Rajalakshmi Engineering College, 2026.
- [20] Rajalakshmi Engineering College, Department of Computer Science and Engineering, “CS23721 - Project Phase I Review Form,” Team T57, 2026.

Project Guide final verification / signature: _____

Final user-testing metrics inserted after approved test run: _____